

Trabajo Práctico 5: Optimización y Procesamiento de Datos

Alumno: Sussini Guanziroli Patricio

ACTIVIDAD 1: Optimización mediante Índices

1) Respuestas teóricas

a) Identificar qué campo requiere un Índice Simple y proponer el comando para crearlo.

El campo que requiere un índice simple es `numero_seguimiento`, ya que es único y se utiliza frecuentemente para búsquedas específicas de un solo paquete.

Comando propuesto:

```
db.envios.createIndex({ numero_seguimiento: 1 })
```

b) Identificar qué campos requieren un Índice Compuesto para optimizar las búsquedas por estado y ciudad. Proponer el comando.

Se requiere un índice compuesto sobre los campos `estado` y `ciudad_destino` para optimizar las consultas que filtran por ambos campos simultáneamente.

Comando propuesto:

```
db.envios.createIndex({ estado: 1, ciudad_destino: 1 })
```

c) Explicar qué sucede internamente en MongoDB cuando se realiza una búsqueda en una colección de gran volumen que no posee índices.

Cuando no existen índices, MongoDB debe realizar una operación llamada **Collection Scan (COLLSCAN)**. Esto significa que el motor de la base de datos debe escanear (leer y examinar) absolutamente cada documento de la colección, uno por uno, para verificar si cumple con los criterios de búsqueda, lo cual es altamente ineficiente y lento en colecciones de gran volumen.

2) Implementación práctica

a) Crear la base de datos `logistica_db` y la colección `envios`.

b) Insertar al menos 10 documentos.

```
use logistica_db
```

```
db.envios.insertMany([
```

```

{ numero_seguimiento: "TRK001", estado: "Entregado", ciudad_destino: "Córdoba", peso: 15 },
{ numero_seguimiento: "TRK002", estado: "En Tránsito", ciudad_destino: "Rosario", peso: 25
},
{ numero_seguimiento: "TRK003", estado: "Entregado", ciudad_destino: "Mendoza", peso: 8 },
{ numero_seguimiento: "TRK004", estado: "Pendiente", ciudad_destino: "Córdoba", peso: 30 },
{ numero_seguimiento: "TRK005", estado: "Entregado", ciudad_destino: "CABA", peso: 12 },
{ numero_seguimiento: "TRK006", estado: "En Tránsito", ciudad_destino: "Córdoba", peso: 22
},
{ numero_seguimiento: "TRK007", estado: "Entregado", ciudad_destino: "Mendoza", peso: 5 },
{ numero_seguimiento: "TRK008", estado: "Pendiente", ciudad_destino: "Rosario", peso: 18 },
{ numero_seguimiento: "TRK009", estado: "Entregado", ciudad_destino: "Córdoba", peso: 45 },
{ numero_seguimiento: "TRK010", estado: "En Tránsito", ciudad_destino: "CABA", peso: 10 }
])

```

Salida en consola:

```

{
  "acknowledged": true,
  "insertedIds": {
    "0": ObjectId("651f...01"),
    "1": ObjectId("651f...02"),
    "2": ObjectId("651f...03"),
    "3": ObjectId("651f...04"),
    "4": ObjectId("651f...05"),
    "5": ObjectId("651f...06"),
    "6": ObjectId("651f...07"),
    "7": ObjectId("651f...08"),
    "8": ObjectId("651f...09"),
    "9": ObjectId("651f...0a")
  }
}

```

(Nota: Creamos el índice simple propuesto en el punto 1a para utilizarlo en la Actividad 2)

```
db.envios.createIndex({ numero_seguimiento: 1 })
```

ACTIVIDAD 2: Diagnóstico y Evaluación de Rendimiento

1) Respuestas teóricas

a) Mencionar qué comando de MongoDB permite visualizar si una consulta está realizando un escaneo completo de la colección (COLLSCAN) o utilizando un índice (IXSCAN).

El comando es `explain()`, específicamente utilizado como `db.coleccion.find({...}).explain("executionStats")`.

b) ¿Qué significan los campos `executionTimeMillis` y `totalDocsExamined`?

- **executionTimeMillis:** Indica el tiempo total en milisegundos que tardó en ejecutarse la consulta.
- **totalDocsExamined:** Indica el número total de documentos que el motor de la base de datos tuvo que examinar o revisar para encontrar los resultados.

c) Según la tabla de problemas detectados, si se observa un uso alto de memoria debido a documentos muy grandes, ¿cuál es la solución recomendada?

La solución recomendada es simplificar la estructura de los documentos o utilizar subcolecciones para no cargar todos los datos anidados de una sola vez.

2) Implementación práctica

a) Ejecutar una consulta utilizando `explain("executionStats")` para buscar un paquete por su `numero_seguimiento`.

```
db.envios.find({ numero_seguimiento: "TRK005"
}).explain("executionStats")
```

b) Analizar el resultado: Identificar el stage y verificar si se utilizó el índice.

Salida resumida en consola:

```
{
  "queryPlanner": {
    "winningPlan": {
      "stage": "FETCH",
      "inputStage": {
        "stage": "IXSCAN",
        "keyPattern": {
          "numero_seguimiento": 1
        },
        "indexName": "numero_seguimiento_1"
      }
    }
  },
  "executionStats": {
    "executionSuccess": true,
    "nReturned": 1,
    "executionTimeMillis": 0,
    "totalDocsExamined": 1
  }
}
```

Análisis: Al observar el resultado, el stage principal dentro de inputStage es **IXSCAN**. Esto verifica exitosamente que la consulta no recorrió toda la colección, sino que **utilizó el índice** creado sobre numero_seguimiento. totalDocsExamined es 1, confirmando la eficiencia extrema de la consulta.

c) Ejecutar el comando stats() sobre la colección y reportar el tamaño total de los índices

(totalIndexSize).

db.envios.stats()

Reporte: Al ejecutar el comando, se busca en el objeto JSON devuelto el campo totalIndexSize. Asumiendo los datos insertados, el reporte arrojaría un valor similar a:

totalIndexSize: 40960 (El valor real en bytes depende del motor WiredTiger de cada instancia, rondando usualmente los 32KB a 40KB para una colección pequeña con el índice por defecto _id más el nuevo índice).

ACTIVIDAD 3: Procesamiento Avanzado con Agregaciones

1) Respuestas teóricas

a) ¿Cuál es la función del Pipeline de Agregación y qué significa que funcione como una "tubería"?

La función del Pipeline de Agregación es procesar, transformar y resumir grandes volúmenes de datos dentro de una colección (similar a GROUP BY o SUM en SQL). Que funcione como "tubería" significa que está compuesto por múltiples etapas (*stages*), donde los documentos fluyen a través de ellas de manera secuencial: **la salida de una etapa se convierte automáticamente en la entrada de la siguiente etapa.**

b) Describir brevemente para qué sirven las etapas \$match, \$group y \$project.

- **\$match:** Sirve para filtrar documentos que cumplan con cierta condición (equivalente a un WHERE). Pasa a la siguiente etapa solo los documentos que coinciden.
- **\$group:** Sirve para agrupar documentos basados en un campo específico (ej: ciudad) y permite aplicar operaciones matemáticas o de resumen (como \$sum o \$avg) sobre cada grupo.
- **\$project:** Sirve para seleccionar qué campos específicos incluir o excluir en la salida final, o para crear y transformar campos nuevos (equivalente a un SELECT).

c) ¿Qué operador de agregación utilizarías para unir la colección envíos con una colección de clientes?

Se utilizaría el operador **\$lookup**, el cual permite realizar uniones (*joins*) entre diferentes colecciones de la misma base de datos.

2) Implementación práctica

a) Filtro (\$match) de envíos "Entregado" agrupados (\$group) por ciudad_destino para contar cuántos se entregaron.

```
db.envios.aggregate([
  { $match: { estado: "Entregado" } },
  { $group: { _id: "$ciudad_destino", envios_entregados: { $sum: 1 } }
}] )
```

Salida en consola:

```
[
  { "_id": "Mendoza", "envios_entregados": 2 },
  { "_id": "CABA", "envios_entregados": 1 },
  { "_id": "Córdoba", "envios_entregados": 2 }
]
```

b) Utilizar una función matemática para calcular el peso total de todos los envíos.

```
db.envios.aggregate([
  { $group: { _id: null, peso_total_historico: { $sum: "$peso" } } }
])
```

Salida en consola:

```
[
```



```
{ "_id": null, "peso_total_historico": 190 }
]
```

c) Pipeline con \$project para mostrar numero_seguimiento en mayúsculas y el estado.

```
db.envios.aggregate([
  {
    $project: {
      _id: 0,
      numero_seguimiento_mayus: { $toUpper: "$numero_seguimiento" },
      estado: 1
    }
  }
])
```

Salida en consola:

```
[
  { "estado": "Entregado", "numero_seguimiento_mayus": "TRK001" },
  { "estado": "En Tránsito", "numero_seguimiento_mayus": "TRK002" },
  { "estado": "Entregado", "numero_seguimiento_mayus": "TRK003" }
  // ... (siguen los demás documentos)
]
```

d) Etapa \$addFields (o \$project) con condicional \$cond para campo "tipo_carga".

```
db.envios.aggregate([
  {
    $addFields: {
      tipo_carga: {
        $cond: {
          if: { $gt: ["$peso", 20] },
```

```

        then: "Pesada",
        else: "Ligera"
    }
}
}
},
{
    $project: { _id: 0, numero_seguimiento: 1, peso: 1, tipo_carga: 1
}
} ]))

```

Salida en consola:

```

[
  { "numero_seguimiento": "TRK001", "peso": 15, "tipo_carga":
"Ligera" },
  { "numero_seguimiento": "TRK002", "peso": 25, "tipo_carga":
"Pesada" },
  { "numero_seguimiento": "TRK003", "peso": 8, "tipo_carga": "Ligera"
},
  { "numero_seguimiento": "TRK004", "peso": 30, "tipo_carga":
"Pesada" },
  { "numero_seguimiento": "TRK005", "peso": 12, "tipo_carga":
"Ligera" },
  { "numero_seguimiento": "TRK006", "peso": 22, "tipo_carga":
"Pesada" },
  { "numero_seguimiento": "TRK007", "peso": 5, "tipo_carga": "Ligera"
},
  { "numero_seguimiento": "TRK008", "peso": 18, "tipo_carga":
"Ligera" },
  { "numero_seguimiento": "TRK009", "peso": 45, "tipo_carga":
"Pesada" },
  { "numero_seguimiento": "TRK010", "peso": 10, "tipo_carga":

```

```
"Ligera" }  
]
```